JAVA PLACEMENT MASTER SERIES · HANDBOOK 5 (FINAL)

# Backend Java & Interview Mastery

*Spring Boot, real project architecture & the complete final interview revision notebook*

★ Java → Spring Boot Bridge

★ DTO, JDBC & Real Architecture

★ Debugging & Output Prediction

★ Top 100 Interview Q&A

---

For MCA / B.Tech / B.E. Placement Preparation

**Backend Developer Roles** · Final Placement Revision

# Table of Contents



Tip: This is the final handbook — it assumes you've studied Handbooks 1–4. If you're short on time, jump straight to **Part 18 (Last-Minute Placement Revision)** and **Part 15 (Top 100 Questions)**.

## How to use this handbook

This is **Handbook 5** — the final handbook in the Java Placement Master Series. It connects everything from Handbooks 1–4 to real backend/Spring Boot development, and closes with an intensive final interview revision: output prediction, debugging scenarios, a Top 100 question bank, HR-technical bridge answers, a 7-day revision plan, and a 15-minute emergency cheat sheet.

Importance tags used throughout:

- ⭐⭐⭐⭐⭐ Must Know for Placements
- ⭐⭐⭐⭐ Frequently Asked
- ⭐⭐ Medium Importance

# Part 1 — Most Important Java Packages ⭐⭐⭐⭐⭐

## java.lang (auto-imported, core language classes)

| Class | Purpose | Real Project Usage |
|---|---|---|
| `Object` | Root of all classes — `equals()`, `hashCode()`, `toString()` | Every entity overrides these for correct collection/logging behavior |
| `String` | Immutable text | Everywhere — API payloads, DB values, logs |
| `Math` | Static math utilities (`Math.max`, `Math.round`, `Math.pow`) | Calculations, pagination math, discount logic |
| `System` | `System.currentTimeMillis()`, `System.exit()`, `System.getenv()` | Timestamps, reading environment config, exit codes in batch jobs |
| `Runtime` | JVM-level info (`availableProcessors()`, `freeMemory()`) | Sizing thread pools dynamically, health-check endpoints |

**Interview Q:** *Why is `equals()` / `hashCode()` in `Object` and not `Comparable`?* — Because equality is a universal concept every object needs (for collections like `HashMap`), while ordering is optional and type-specific.

## java.util

| Class | Purpose | Real Project Usage |
|---|---|---|
| `Collections` | Static utility methods ( `sort` , `unmodifiableList` ) | Wrapping repository results as read-only lists |
| `Optional` | Explicit "maybe absent" wrapper | Repository `findById()` return types |
| `Comparator` | Custom sorting | Multi-field sort in product/order listings |
| `Objects` | Null-safe helpers ( `Objects.equals` , `Objects.requireNonNull` , `Objects.hash` ) | Null-safe `equals()` / `hashCode()` implementations, guard clauses |

```
// Objects.requireNonNull - fail-fast guard clause, very common in constructors
public Order(Customer customer) {
    this.customer = Objects.requireNonNull(customer, "customer must not be null");
}
```

## java.util.stream — functional-style data processing (see Part 9 for backend examples)

## java.util.function — `Predicate` , `Function` , `Consumer` , `Supplier` (the building blocks behind Streams and validation logic)

## java.util.concurrent — thread pools, `ConcurrentHashMap` , `CompletableFuture` (Handbook 4 covers this deeply; backend usage in Part 5/12)

## java.time — modern Date/Time API

| Class | Real Project Usage |
|---|---|
| `LocalDate` / `LocalDateTime` | Order dates, appointment scheduling |
| `Instant` | Precise event timestamps (logs, audit trails) |
| `Duration` / `Period` | SLA calculations, subscription periods |
| `DateTimeFormatter` | Formatting dates for API responses |

## java.io & java.nio — file/stream handling

Used for file upload/download features, report generation (CSV/PDF export), reading configuration files. Modern code prefers NIO's `Path` / `Files` API (Handbook 2, Part 8).

## java.util.regex — `Pattern` / `Matcher`

Used for input validation (email, phone, password format) at the DTO/request-validation layer.

## java.math — `BigDecimal`, `BigInteger`

**Critical for backend interviews ⭐⭐⭐⭐⭐:** Never use `double` / `float` for money — binary floating point can't represent many decimal fractions exactly, causing rounding errors. Always use `BigDecimal` for financial calculations.

```
BigDecimal price = new BigDecimal("19.99");       // ALWAYS construct from String, not double!
BigDecimal total = price.multiply(BigDecimal.valueOf(3));
```

## java.sql (interview level)

| Interface | Purpose |
|---|---|
| `Connection` | Represents a DB connection |
| `PreparedStatement` | Parameterized, precompiled SQL — prevents SQL injection (Part 10) |
| `ResultSet` | Iterates query results row by row |

In modern Spring Boot apps, Spring Data JPA/Hibernate abstracts most raw JDBC away — but interviewers still expect you to understand what happens underneath (Part 10).

### Interview Questions — Part 1

**Q1. Why should BigDecimal always be constructed from a String, not a double?** ⭐⭐⭐⭐⭐ Frequently Asked

- **Ideal Answer:** `double` itself already lost precision representing the decimal value in binary — constructing `BigDecimal` from that flawed `double` bakes in the error. Constructing from the exact decimal `String` avoids this entirely.
- **Difficulty:** Intermediate

**Q2. What's the real-world risk of using Optional as a repository return type incorrectly?** ⭐⭐⭐⭐

- **Ideal Answer:** Calling `.get()` without checking presence defeats the purpose — the right pattern is `.orElseThrow()` or `.map()` / `.orElse()` to force explicit handling of the "not found" case.
- **Difficulty:** Basic

# Part 2 — Java Design Principles

| Principle | Meaning | Interview Sound Bite |
|---|---|---|
| **DRY** (Don't Repeat Yourself) | Avoid duplicating logic — extract shared code into one place | "Duplication means a bug fix needs to happen in N places instead of one" |
| **KISS** (Keep It Simple, Stupid) | Prefer the simplest solution that works over clever complexity | "Simple code is easier to debug, test, and hand off to teammates" |
| **YAGNI** (You Aren't Gonna Need It) | Don't build functionality "just in case" — build what's actually needed now | "Speculative generality adds maintenance cost for features that may never be used" |
| **Composition over Inheritance** | Prefer object composition (HAS-A) over deep inheritance hierarchies | "Inheritance creates tight coupling to a parent's implementation; composition is flexible and swappable at runtime" |
| **Favor Immutability** | Prefer objects that can't change state after creation | "Immutable objects are inherently thread-safe and easier to reason about" |

```
// YAGNI violation: over-engineered "just in case" flexibility nobody asked for
interface PaymentStrategy { void pay(); }   // fine IF multiple payment types are actually needed NOW
// but adding this for a single hardcoded payment type "in case we need more later" is YAGNI
```

# Part 3 — SOLID Principles (Interview Focus) ⭐⭐⭐⭐⭐

| Principle | One-Line Definition | Java Example | Spring Boot Example | Common Mistake |
|---|---|---|---|---|
| **S** — Single Responsibility | A class should have one reason to change | Separate `OrderValidator` from `OrderRepository` | `Controller` only handles HTTP; `Service` only handles business logic | Fat "God classes" doing validation + persistence + business logic together |
| **O** — Open/Closed | Open for extension, closed for modification | New `Shape` subclasses don't require editing `AreaCalculator` | New `PaymentMethod` implementations don't require editing `PaymentService` | Adding `if/else` / `switch` chains for every new type instead of polymorphism |
| **L** — Liskov Substitution | Subtypes must be substitutable for their base type | A `Square extends Rectangle` that breaks `setWidth()` violates this | A custom `UserDetails` implementation must honor Spring Security's expected contract | Overriding a method to throw `UnsupportedOperationException`, breaking callers' expectations |
| **I** — Interface Segregation | Prefer small, specific interfaces over one large one | Split `Worker` into `Payable`, `Schedulable` | Separate `ReadRepository` / `WriteRepository` instead of one bloated repository interface | One giant `Service` interface forcing unrelated implementations to stub unused methods |
| **D** — Dependency Inversion | Depend on abstractions, not concrete classes | `Service` depends on a `Repository` interface | `@Autowired` a `UserService` interface, not `UserServiceImpl` directly | `new UserServiceImpl()` hardcoded inside a controller instead of injecting the interface |

**Interview explanation ⭐⭐⭐⭐⭐:** "SOLID principles guide you toward code that's easy to extend without breaking existing functionality — which is exactly what Spring Boot's dependency injection and interface-driven design encourage by default."

# Part 4 — Immutability & Thread Safety ⭐⭐⭐⭐⭐

## Creating an Immutable Class

```
final class Money {                              // 1. class is final - no subclassing
    private final BigDecimal amount;               // 2. all fields are final
    private final String currency;

    public Money(BigDecimal amount, String currency) {
        this.amount = amount;                        // 3. set once, in the constructor
        this.currency = currency;
    }

    public BigDecimal getAmount() { return amount; }   // 4. only getters, NO setters
    public String getCurrency() { return currency; }

    public Money add(Money other) {                    // 5. "mutating" methods return a NEW instance
        return new Money(this.amount.add(other.amount), this.currency);
    }
}
```

**Rules for immutability ⭐⭐⭐⭐⭐:**

1. Make the class `final` (prevent subclasses from breaking immutability)
2. All fields `private final`
3. No setters
4. If a field is a mutable object (e.g., a `List` or `Date`), return a **defensive copy** from getters, never the original reference
5. Any "modification" method returns a brand-new instance instead of changing state

## Why It Matters — The Thread Safety Connection ⭐⭐⭐⭐⭐

An object with **no mutable state** cannot have a race condition — there's nothing for two threads to corrupt. This is why immutable objects need **zero synchronization** to be safely shared across threads.

## String and Records

- `String` is immutable for exactly these reasons — safe String Pool sharing, thread safety, cached hashcode (Handbook 2, Part 2).
- `record` (Java 16+) gives you immutability **for free** — fields are implicitly `private final`, no setters are generated, and the class is implicitly `final`.

```
record Money(BigDecimal amount, String currency) {}   // immutable, thread-safe, in one line
```

### Interview Questions — Part 4

**Q1. Why are immutable objects thread-safe by default?** ⭐⭐⭐⭐⭐ Frequently Asked

- **Ideal Answer:** Thread-safety problems arise from concurrent modification of shared mutable state. An immutable object's state never changes after construction, so there's nothing to corrupt — multiple threads can safely read it simultaneously with no synchronization needed.
- **Difficulty:** Intermediate

**Q2. If an immutable class has a mutable field like a List, is it truly immutable?** ⭐⭐⭐⭐⭐

- **Ideal Answer:** Only if the class defends against external mutation — the constructor should store a defensive copy of the list, and the getter should return an unmodifiable view (or another copy), otherwise callers can mutate the "immutable" object's internal list directly.
- **Why asked:** A classic trap that catches candidates who only apply the surface-level rules (final fields, no setters) without considering mutable field references.
- **Difficulty:** Advanced

**Q3. How do Records simplify immutability?** ⭐⭐⭐⭐

- **Ideal Answer:** Records automatically generate a final class with private final fields, accessor methods, `equals()`, `hashCode()`, and `toString()` — giving you a correctly implemented immutable data carrier without writing any boilerplate.
- **Difficulty:** Basic

# Part 5 — Java in Spring Boot

## The Request Flow

## Core Concepts, Mapped to Core Java

| Spring Concept | Core Java Concept Behind It |
|---|---|
| **Dependency Injection (DI)** | Composition — a class "has-a" dependency handed to it, instead of creating it with `new` |
| **Inversion of Control (IoC)** | The Spring container creates and manages objects instead of your code doing it — control is "inverted" |
| **Bean** | Just an object instance managed by the Spring container (created via a normal Java class) |
| `@Component` / `@Service` / `@Repository` | Stereotype annotations — read via Reflection at startup to register classes as beans |
| `@Controller` | A class handling HTTP requests, backed by annotations processed via Reflection |
| `Entity` | A plain Java class (POJO) mapped to a DB table via JPA annotations |
| `DTO` | A plain class used to transfer data across layers — pure abstraction, decoupling API shape from DB structure |
| **Constructor Injection** ⭐⭐⭐⭐⭐ | The **preferred** DI style — dependencies passed via constructor parameters, enabling `final` fields (immutability!) and easier testing |

```
@Service
public class OrderService {
    private final OrderRepository repository;      // final - immutable, thread-safe reference

    public OrderService(OrderRepository repository) {  // constructor injection (preferred over @Autowired
field injection)
        this.repository = repository;
    }
}
```

**Why constructor injection is preferred ⭐⭐⭐⭐⭐:** Enables `final` fields (immutability), makes dependencies explicit and mandatory (can't construct the object without them), and is far easier to unit test (just call the constructor directly with mocks — no need for a Spring context or reflection-based field injection).

## Where Core Java Concepts Show Up

| Core Java Concept | Where It Appears in Spring Boot |
|---|---|
| Interfaces & Polymorphism | `Repository` / `Service` interfaces with Spring-generated or custom implementations |
| Collections | `List<T>` / `Map<K,V>` in request/response bodies, query results |
| Exceptions | Custom exceptions mapped to HTTP responses via `@ExceptionHandler` (Part 6) |
| Streams | Transforming `List<Entity>` → `List<DTO>` in service layers (Part 9) |
| Generics | `JpaRepository<T, ID>`, `ResponseEntity<T>` |
| Optional | `findById()` returning `Optional<Entity>` |

# Part 6 — Exception Handling in APIs

## Global Exception Handling

```
@RestControllerAdvice                          // applies across ALL controllers
public class GlobalExceptionHandler {

    @ExceptionHandler(UserNotFoundException.class)
    public ResponseEntity<ErrorResponse> handleNotFound(UserNotFoundException ex) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND)
            .body(new ErrorResponse("USER_NOT_FOUND", ex.getMessage()));
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)   // validation errors
    public ResponseEntity<ErrorResponse> handleValidation(MethodArgumentNotValidException ex) {
        String message = ex.getBindingResult().getFieldErrors().stream()
            .map(err -> err.getField() + ": " + err.getDefaultMessage())
            .collect(Collectors.joining(", "));
        return ResponseEntity.badRequest().body(new ErrorResponse("VALIDATION_ERROR", message));
    }

    @ExceptionHandler(Exception.class)                          // catch-all fallback
    public ResponseEntity<ErrorResponse> handleGeneric(Exception ex) {
        return ResponseEntity.internalServerError()
            .body(new ErrorResponse("INTERNAL_ERROR", "Something went wrong"));   // generic message!
    }
}
```

**Why one centralized handler ⭐⭐⭐⭐⭐:** Avoids duplicating try-catch blocks in every controller method (DRY), guarantees consistent error response shape across the whole API, and keeps controllers focused only on request handling.

## Custom Exceptions

```
public class UserNotFoundException extends RuntimeException {   // unchecked - doesn't clutter method
signatures
    public UserNotFoundException(Long id) {
        super("User not found with id: " + id);
    }
}
```

## Why NOT Expose Stack Traces ⭐⭐⭐⭐⭐

Returning a raw stack trace in an API response is a **security risk** — it leaks internal package structure, library versions, and sometimes even query details, giving attackers a roadmap. Always return a **generic, safe message** to the client, while logging the **full details internally** (with a correlation/trace ID the client can reference for support).

```
log.error("Unexpected error [traceId={}]", traceId, ex);       // full detail, server-side only
return ResponseEntity.internalServerError()
    .body(new ErrorResponse("INTERNAL_ERROR", "Something went wrong. Reference: " + traceId));  // safe,
client-facing
```

### Interview Questions — Part 5–6

**Q1. Why is constructor injection preferred over field injection in Spring? ⭐⭐⭐⭐⭐** Frequently Asked

- **Ideal Answer:** It allows dependencies to be `final` (immutable, thread-safe references), makes required dependencies explicit at construction time (fails fast if missing), and is far easier to unit test since you can instantiate the class directly with mocks, without needing a Spring context.
- **Difficulty:** Intermediate

**Q2. Why should you never return a raw stack trace in an API error response? ⭐⭐⭐⭐⭐**

- **Ideal Answer:** It's a security risk — exposing internal implementation details (package names, library versions, sometimes query fragments) gives attackers useful reconnaissance information. Always log full details server-side and return a generic, safe error message to the client.
- **Difficulty:** Basic

**Q3. What is `@RestControllerAdvice` and why use it over per-controller try-catch? ⭐⭐⭐⭐⭐**

- **Ideal Answer:** It centralizes exception handling across all controllers in one place, avoiding duplicated try-catch logic (DRY) and guaranteeing a consistent error response format across the entire API.
- **Difficulty:** Intermediate

# Part 7 — DTO vs Entity

## Why DTOs Exist

An **Entity** maps directly to a database table — it's shaped by your **schema**. A **DTO** (Data Transfer Object) is shaped by what the **API** needs to expose. Using the Entity directly as your API response couples your database structure to your public API contract — a design mistake that bites you later.

| Reason | Explanation |
|---|---|
| **Security** | Entities often contain sensitive fields (password hash, internal flags) that should never be serialized into an API response |
| **Performance** | DTOs can flatten/trim data, avoiding accidentally serializing entire lazy-loaded object graphs (a classic Hibernate `LazyInitializationException` /N+1 trap) |
| **API Stability** | You can change your DB schema without breaking API consumers, as long as the DTO mapping absorbs the change |
| **Decoupling** | Multiple DTOs can present the same Entity differently for different use cases (e.g., `UserSummaryDTO` vs `UserDetailDTO` ) |

## Comparison Table

| | Entity | DTO |
|---|---|---|
| Shaped by | Database schema ( `@Entity` , `@Table` , `@Column` ) | API contract / client needs |
| Contains | All persisted fields, including sensitive/internal ones | Only what's safe and needed for a specific use case |
| Lifecycle | Tied to persistence context (JPA managed) | Plain object, no persistence behavior |
| Mutability | Often mutable (JPA requires a no-arg constructor + setters) | Often immutable (record is a great fit) |
| Direction | Internal — service/repository layer | External — controller/API boundary |

## Mapping

```
// Manual mapping (simple, explicit, no extra dependency)
public UserDTO toDTO(User user) {
    return new UserDTO(user.getName(), user.getEmail());   // password hash NOT included - safe by
construction
}

// Streams-based bulk mapping (Part 9)
List<UserDTO> dtos = users.stream().map(this::toDTO).collect(Collectors.toList());
```

For larger projects, libraries like **MapStruct** generate this mapping code at compile time, avoiding both boilerplate and reflection overhead.

### Interview Questions — Part 7

**Q1. Why not just return the Entity directly from a REST controller?** ⭐⭐⭐⭐⭐ Frequently Asked

- **Ideal Answer:** It couples your API contract to your database schema (a schema change breaks API consumers), risks leaking sensitive fields, and can trigger lazy-loading serialization issues. A DTO layer decouples these concerns and gives explicit control over what's exposed.
- **Difficulty:** Intermediate

# Part 8 — Optional in Real Projects ⭐⭐⭐⭐⭐

## Repository Layer

```
public interface UserRepository extends JpaRepository<User, Long> {
    Optional<User> findByEmail(String email);   // Spring Data JPA convention - returns Optional
}
```

## Service Layer — Turning "Maybe Absent" into a Decision

```
public User getUserByEmail(String email) {
    return userRepository.findByEmail(email)
        .orElseThrow(() -> new UserNotFoundException(email));   // explicit decision: not found = error
}

public String getUserDisplayName(String email) {
    return userRepository.findByEmail(email)
        .map(User::getName)
        .orElse("Guest");                                       // explicit decision: not found = default
}
```

**Why this avoids NullPointerException** ⭐⭐⭐⭐⭐**:** The compiler and the API itself **force** you to decide what happens in the empty case at the call site — you can't accidentally forget a null check three method calls later, because there's no raw `null` floating around unexamined.

## Common Misuse in Real Projects

| Misuse | Why It's Wrong | Fix |
| --- | --- | --- |
| `Optional<User>` as an **entity field** | Not serializable in the way JPA/Jackson expect; adds pointless wrapping overhead | Use `Optional` only as a method **return type**, never a field |
| Calling `.get()` immediately after `findById()` | Defeats the entire purpose — can still throw | Use `.orElseThrow()` with a meaningful custom exception |
| `Optional<List<T>>` | Redundant — just return an **empty list** | Return `List.of()` instead of wrapping in Optional |
| Overusing `.orElse(expensiveCall())` | Eagerly evaluates the fallback even when not needed | Use `.orElseGet(() -> expensiveCall())` |

# Part 9 — Streams in Backend Development

⭐⭐⭐⭐⭐

## Practical Examples

```
// Filtering + Mapping - classic service-layer DTO conversion
List<ProductDTO> activeProducts = products.stream()
    .filter(Product::isActive)
    .map(this::toDTO)
    .collect(Collectors.toList());

// Grouping - dashboard/report style aggregation
Map<String, List<Order>> ordersByStatus = orders.stream()
    .collect(Collectors.groupingBy(Order::getStatus));

// Aggregation - sum, average, count
BigDecimal totalRevenue = orders.stream()
    .map(Order::getAmount)
    .reduce(BigDecimal.ZERO, BigDecimal::add);

double avgOrderValue = orders.stream()
    .mapToDouble(o -> o.getAmount().doubleValue())
    .average()
    .orElse(0.0);

// Sorting - multi-field, common in listing endpoints
List<Product> sorted = products.stream()
    .sorted(Comparator.comparing(Product::getCategory)
                      .thenComparing(Product::getPrice, Comparator.reverseOrder()))
    .collect(Collectors.toList());

// Pagination support - combine with a repository's Pageable in Spring Data, or manually:
List<Product> page = products.stream()
    .skip((long) pageNumber * pageSize)
    .limit(pageSize)
    .collect(Collectors.toList());
```

> ***Real pagination note*** ⭐⭐⭐⭐**:** *In production, prefer* ***database-level pagination*** *(* `Pageable` *in Spring Data JPA) over in-memory* `skip` */* `limit` *— streaming an entire table into memory just to paginate it doesn't scale.*

## When Streams Reduce Readability ⭐⭐⭐⭐

| Prefer a loop when... | Prefer a Stream when... |
|---|---|
| Logic has multiple side effects/complex branching | Logic is a clear, linear transformation pipeline |
| You need early exit with complex conditions | You're filtering/mapping/collecting straightforwardly |
| Debugging step-by-step is important | The pipeline reads naturally top-to-bottom |
| Deeply nested streams reduce clarity | A single, flat chain stays readable |

```
// Reduces readability - deeply nested, hard to debug
result = list.stream().filter(...).map(x -> x.stream().filter(...).map(...).collect(...)).collect(...);
// A plain nested loop is often clearer here
```

### Interview Questions — Part 8–9

**Q1. Why shouldn't Optional be used as an entity field? ⭐⭐⭐⭐**

- **Ideal Answer:** JPA entities need standard getters/setters and no-arg constructors for ORM mapping, and `Optional` isn't designed or intended for that role — it adds serialization complications for no real benefit. Use it only as a method return type.
- **Difficulty:** Intermediate

**Q2. When would you avoid using Streams in a service method? ⭐⭐⭐⭐**

- **Ideal Answer:** When the logic involves complex branching, multiple side effects, or deep nesting that would make a stream chain harder to read and debug than an equivalent plain loop — readability should win over "using streams for everything."
- **Difficulty:** Intermediate

# Part 10 — Java & Databases (Interview Level) ⭐⭐⭐⭐⭐

## JDBC Flow

```
1. Load driver (auto since JDBC 4.0)
2. DriverManager.getConnection(url, user, pass)  ⟶  Connection
3. connection.prepareStatement(sql)              ⟶  PreparedStatement
4. statement.executeQuery() / executeUpdate()    ⟶  ResultSet (for queries)
5. Iterate ResultSet, map rows to objects
6. Close resources (try-with-resources!)
```

```
String sql = "SELECT id, name FROM users WHERE email = ?";
try (Connection conn = dataSource.getConnection();
     PreparedStatement stmt = conn.prepareStatement(sql)) {
    stmt.setString(1, email);
    try (ResultSet rs = stmt.executeQuery()) {
        while (rs.next()) {
            System.out.println(rs.getLong("id") + " " + rs.getString("name"));
        }
    }
}
```

## PreparedStatement ⭐⭐⭐⭐⭐ — SQL Injection Prevention

```
// VULNERABLE - string concatenation, classic SQL injection hole
String badSql = "SELECT * FROM users WHERE email = '" + userInput + "'";
// If userInput = "' OR '1'='1", this returns ALL users!

// SAFE - PreparedStatement treats the parameter as DATA, never as executable SQL
String safeSql = "SELECT * FROM users WHERE email = ?";
stmt.setString(1, userInput);   // parameter is escaped/typed, can never alter the query structure
```

**Why it matters ⭐⭐⭐⭐⭐:** `PreparedStatement` pre-compiles the SQL structure **before** binding parameters — user input is always treated as a literal value, never as part of the SQL syntax itself, structurally preventing injection (unlike manually escaping strings, which is error-prone).

## Batch Updates ⭐⭐⭐⭐

Executing many inserts/updates one-by-one means one network round-trip per statement — slow. **Batching** groups them into fewer round-trips.

```
try (PreparedStatement stmt = conn.prepareStatement("INSERT INTO logs (msg) VALUES (?)")) {
    for (String msg : messages) {
        stmt.setString(1, msg);
        stmt.addBatch();              // queue instead of executing immediately
    }
    stmt.executeBatch();               // sends all queued statements together
}
```

## Connection Pooling (Concept) ⭐⭐⭐⭐

Opening a new DB connection is **expensive** (TCP handshake, authentication). A **connection pool** (e.g., HikariCP, Spring Boot's default) keeps a set of open, reusable connections ready to hand out — dramatically reducing per-request connection overhead.

```
Request comes in → borrow a connection from the pool → use it → return it to the pool (not closed!)
```

### Interview Questions — Part 10

**Q1. How does PreparedStatement prevent SQL injection?** ⭐⭐⭐⭐⭐ Frequently Asked

- **Ideal Answer:** The SQL query structure is compiled first, with placeholders (`?`) for parameters. Bound values are always treated as literal data, never re-parsed as SQL syntax — so malicious input can't alter the query's structure, unlike string concatenation.
- **Difficulty:** Basic

**Q2. Why use connection pooling instead of opening a new connection per request?** ⭐⭐⭐⭐⭐

- **Ideal Answer:** Establishing a DB connection is expensive (network handshake, authentication). A pool keeps a set of connections open and ready for reuse, so requests "borrow" and "return" connections instead of paying that setup cost every time — dramatically improving throughput under load.
- **Difficulty:** Intermediate

# Part 11 — Real Project Architecture ⭐⭐⭐⭐⭐

## E-Commerce Package Structure

```
com.example.ecommerce
 ├── controller/       → OrderController, ProductController       (HTTP layer)
 ├── service/          → OrderService, ProductService               (business logic)
 ├── repository/       → OrderRepository, ProductRepository           (data access, extends JpaRepository)
 ├── entity/           → Order, Product, Customer                     (JPA-mapped DB classes)
 ├── dto/              → OrderDTO, ProductDTO, CreateOrderRequest      (API-facing shapes)
 ├── exception/        → OrderNotFoundException, GlobalExceptionHandler
 ├── config/           → SecurityConfig, SwaggerConfig, ThreadPoolConfig
 └── util/             → DateUtils, PriceCalculator                   (stateless helper classes)
```

## Request Flow — Placing an Order

```
POST /api/orders
      |
      ▼
OrderController.createOrder(@RequestBody @Valid CreateOrderRequest request)
      |  (Bean Validation runs here - annotations like @NotNull, @Positive)
      ▼
OrderService.createOrder(request)
      |  - maps DTO → Entity
      |  - business rules (stock check, pricing)
      |  - calls repository (Optional-based lookups)
      |  - may throw custom exceptions (caught by @RestControllerAdvice)
      ▼
OrderRepository.save(order)
      |  (Spring Data JPA generates the SQL via Hibernate)
      ▼
Database
      |
      ▼ (result flows back up)
OrderService maps Entity → OrderDTO (streams .map())
      |
      ▼
OrderController returns ResponseEntity<OrderDTO>
```

## Where Each Java Concept Shows Up

| Layer | Java/Handbook Concepts Used |
|---|---|
| Controller | Annotations (Reflection-based), generics ( `ResponseEntity<T>` ) |
| Service | Interfaces, OOP, Streams, Optional, custom exceptions, `BigDecimal` for pricing |
| Repository | Generics ( `JpaRepository<T, ID>` ), Optional return types |
| Entity | Encapsulation, `equals()` / `hashCode()` , JPA annotations |
| DTO | Records (immutability), mapping via Streams |
| Exception | Custom exception hierarchy, `@RestControllerAdvice` |
| Config | Dependency Injection, Bean definitions, thread pool sizing (Handbook 4) |
| Util | Static methods, `java.time` , `BigDecimal` math |

# Part 12 — Frequently Asked Backend Scenarios ⭐⭐⭐⭐⭐

### 1. Design a login API

- **Approach:** `POST /api/login` accepts email/password DTO → Service verifies credentials against a hashed password (BCrypt) → issues a JWT/session token on success.
- **Java concepts:** DTO validation, `Optional` for user lookup, exception handling for invalid credentials.
- **Spring concepts:** `@RestController`, Spring Security for password hashing/auth, `@ExceptionHandler` for auth failures.
- **Common mistakes:** Storing plaintext passwords; returning different error messages for "wrong email" vs "wrong password" (leaks which emails are registered — should return one generic message).

### 2. Handle duplicate email registration

- **Approach:** Add a unique DB constraint on `email`; catch the resulting `DataIntegrityViolationException` (or pre-check via `existsByEmail()`) and return a clear `409 Conflict`.
- **Java concepts:** Custom exception, `Optional` / `boolean` repository check.
- **Spring concepts:** `@ExceptionHandler`, unique constraint on the `@Entity`.
- **Common mistakes:** Relying only on a pre-check without a DB constraint — a race condition between two concurrent signups can still insert duplicates.

### 3. Paginate products

- **Approach:** Use Spring Data's `Pageable` / `Page<T>` — database-level pagination, not in-memory streaming.
- **Java concepts:** Generics ( `Page<Product>` ).
- **Spring concepts:** `repository.findAll(Pageable pageable)`.
- **Common mistakes:** Fetching the entire table then paginating with Streams `.skip()/.limit()` — doesn't scale.

### 4. Sort products by price

- **Approach:** Pass a `Sort` object into the repository call, or use `Comparator` chains for in-memory lists.
- **Java concepts:** `Comparator.comparing(Product::getPrice)`.
- **Spring concepts:** `Sort.by("price").ascending()` combined with `Pageable`.
- **Common mistakes:** Sorting in application code after fetching an unsorted, unbounded result set from the DB.

### 5. Validate an incoming request

- **Approach:** Annotate DTO fields with Bean Validation ( `@NotNull`, `@Email`, `@Positive` ), add `@Valid` on the controller parameter — validation runs automatically before the method body executes.
- **Java concepts:** Annotations, Reflection (behind the scenes).
- **Spring concepts:** `MethodArgumentNotValidException` handled globally (Part 6).
- **Common mistakes:** Manually re-validating with scattered `if` checks instead of using Bean Validation consistently.

### 6. Handle concurrent order placement (e.g., limited stock)

- **Approach:** Use a DB-level pessimistic lock ( `SELECT ... FOR UPDATE` ) or optimistic locking ( `@Version` field) on the stock row during the check-and-decrement, or a `synchronized` / `ReentrantLock` if it's an in-memory-only resource.
- **Java concepts:** Concurrency (Handbook 4) — race conditions, atomic operations.
- **Spring concepts:** `@Transactional` , `@Version` for optimistic locking.
- **Common mistakes:** Checking stock and decrementing it in two separate, non-atomic steps — a classic race condition ("check-then-act").

**7. Cache frequently accessed data**

- **Approach:** Use `@Cacheable` (Spring Cache abstraction) backed by Redis/Caffeine for read-heavy, rarely-changing data (e.g., product catalog).
- **Java concepts:** `ConcurrentHashMap` (for simple in-memory caches), immutability of cached objects.
- **Spring concepts:** `@Cacheable` , `@CacheEvict` on updates.
- **Common mistakes:** Caching data that changes frequently without a proper eviction/invalidation strategy, leading to stale reads.

**8. Return proper error responses**

- **Approach:** Consistent JSON error shape ( `errorCode` , `message` , `timestamp` ) via `@RestControllerAdvice` , correct HTTP status codes ( `400` validation, `404` not found, `409` conflict, `500` unexpected).
- **Java concepts:** Custom exception hierarchy.
- **Spring concepts:** `@ExceptionHandler` , `ResponseEntity` .
- **Common mistakes:** Returning `200 OK` with an error message in the body instead of the correct status code; inconsistent error response shapes across different endpoints.

## Part 13 — Output Prediction Round

**1.** `String a = "cat"; String b = "cat"; System.out.println(a == b);` **Answer:** `true` — both literals resolve to the same pooled String object.

**2.** `String a = new String("cat"); String b = "cat"; System.out.println(a == b);` **Answer:** `false` — `new String()` creates a separate heap object outside the pool.

**3.** `List<Integer> list = List.of(1,2,3); list.add(4);` **Answer:** Throws `UnsupportedOperationException` — `List.of()` returns an immutable list.

**4.** A class overrides `equals()` but not `hashCode()` ; two "equal" objects are put into a `HashSet` . How many elements? **Answer:** 2 — inconsistent hashCode means they likely land in different buckets, so the duplicate isn't detected.

**5.** `Dog extends Animal` , both override `speak()` . `Animal a = new Dog(); a.speak();` — whose method runs? **Answer:** `Dog` 's — dynamic method dispatch resolves based on the actual object type at runtime.

**6.** A subclass overloads (not overrides) a parent method with a different parameter type. `Parent p = new Child(); p.method(someArg);` — which runs? **Answer:** The `Parent` 's version — overload resolution is based on the **reference type** at compile time, not the runtime object type.

**7.** `try { return 1; } finally { return 2; }` — what does the method return? **Answer:** `2` — a `return` inside `finally` overrides/suppresses the `try` block's return.

**8.** `Stream.of(1,2,3).filter(n -> n > 5).findFirst().get();` **Answer:** Throws `NoSuchElementException` — the Optional returned by `findFirst()` is empty since nothing matches.

**9.** `Optional<String> opt = Optional.ofNullable(null); System.out.println(opt.orElse("default"));` **Answer:** `default` — `ofNullable(null)` produces an empty Optional, so `orElse` returns the fallback.

**10.** `static int x = 5; static { x += 10; System.out.println(x); }` — what prints when the class loads? **Answer:** `15` — static fields/blocks execute top-to-bottom in source order.

**11.** `Integer a = 127, b = 127; Integer c = 128, d = 128; System.out.println((a==b) + " " + (c==d));` **Answer:** `true false` — Integer cache covers -128 to 127 only.

**12.** `final` local variable captured in a lambda is reassigned after the lambda is defined — compiles? **Answer:** No — compile error; captured local variables must be effectively final.

**13.** Two threads increment a shared non-volatile, non-synchronized `int` 100,000 times each. Final value? **Answer:** Unpredictable, usually less than 200,000 — lost updates from the race condition.

**14.** `List<Integer> list = new ArrayList<>(List.of(1,2,3)); for (int i : list) { if (i == 2) list.remove(Integer.valueOf(2)); }` **Answer:** Throws `ConcurrentModificationException` — structural modification during fail-fast iteration.

**15.** `Object obj = "hello"; if (obj instanceof Integer i) { ... } else { System.out.println("not an integer"); }` **Answer:** Prints `"not an integer"` — pattern matching `instanceof` simply fails the type check and falls to `else`, no exception.

## Part 14 — Debugging Round

★★★★★

**1. NullPointerException on `user.getAddress().getCity()`**

- **Root cause:** `getAddress()` returned `null`.
- **Fix:** Null-check each step, or better, make `getAddress()` return `Optional<Address>` and chain with `.map()`.

**2. ConcurrentModificationException while removing from a List in a for-each loop**

- **Root cause:** Structural modification during fail-fast iteration.
- **Fix:** Use `Iterator.remove()`, or `list.removeIf(condition)`.

**3. ClassCastException when casting an Object from a raw-typed collection**

- **Root cause:** A raw `List` (no generic type) allowed an incompatible object to be inserted; the cast fails later, far from the real bug.
- **Fix:** Never use raw types — always declare the generic type (`List<String>`), letting the compiler catch the mismatch at compile time.

**4. StackOverflowError in a recursive method**

- **Root cause:** Missing or incorrect base case causes unbounded recursion.
- **Fix:** Add/correct the base case; consider converting to an iterative solution for deep recursion.

**5. Memory leak via a growing static Map that's never cleared**

- **Root cause:** Entries stay reachable forever via the static reference, so GC can never reclaim them, even after they're logically unused.
- **Fix:** Use a bounded cache (LRU via `LinkedHashMap`, or a proper caching library with eviction) instead of an unbounded static collection.

**6. Infinite recursion causing the app to hang, not crash**

- **Root cause:** A recursive call with no progress toward a base case, but tail-call-like enough or shallow enough not to immediately overflow (e.g., mutual recursion between two methods).
- **Fix:** Trace the call chain, ensure each recursive call moves measurably closer to termination.

**7. Wrong equals()/hashCode() causing HashMap lookups to silently fail**

- **Root cause:** `equals()` overridden but `hashCode()` wasn't (or vice versa) — objects that are "equal" hash to different buckets.
- **Fix:** Always override both together, consistently (based on the same fields), ideally via `Objects.equals()` / `Objects.hash()`.

**8. A Stream pipeline's `peek()` seems to not run at all**

- **Root cause:** `peek()` is intermediate/lazy — if the terminal operation short-circuits before reaching that element (or there's no terminal operation at all), `peek()` never executes for those elements.
- **Fix:** Ensure a terminal operation is present and actually reaches the elements in question; don't rely on `peek()` for essential side effects — use `forEach()` for that.

**9. Optional misuse: `optional.get()` throws NoSuchElementException in production**

- **Root cause:** `.get()` was called without checking `.isPresent()` — defeats Optional's entire purpose.
- **Fix:** Replace with `.orElseThrow(customException)` or `.orElse(default)` / `.map()` chains.

**10. Thread safety bug: a "thread-safe" class using `volatile` still produces incorrect results under load**

- **Root cause:** `volatile` only guarantees visibility, not atomicity — a compound operation like `count++` (read-modify-write) is still a race condition even with `volatile`.
- **Fix:** Use `AtomicInteger` / `AtomicLong` for atomic compound operations, or `synchronized` / `ReentrantLock` around the critical section.

## Part 15 — Top 100 Java Interview Questions ★★★★★

*(Concise model answers — expand using Handbooks 1–4 for full depth.)*

## Basics (20)

1. **What is Java?** — A platform-independent, object-oriented language that compiles to bytecode run by the JVM, known for portability and a mature ecosystem.
2. **JDK vs JRE vs JVM?** — JVM executes bytecode; JRE = JVM + libraries; JDK = JRE + dev tools. Each is a superset of the previous.
3. **Why is Java platform-independent?** — Source compiles to platform-neutral bytecode; only the JVM implementation itself is platform-specific.
4. **What is bytecode?** — Intermediate, platform-neutral compiled code executed by the JVM.
5. **Primitive vs reference types?** — Primitives store actual values directly; references store heap addresses to objects.
6. **What is autoboxing?** — Automatic conversion between a primitive and its wrapper class, e.g., `int` → `Integer`.
7. **Why does Integer caching cause `==` surprises?** — Values -128 to 127 are cached and reused; outside that range, `==` compares distinct objects.
8. **Widening vs narrowing casting?** — Widening (small→big type) is implicit and safe; narrowing is explicit and may lose data.
9. **What is `var`?** — Local variable type inference; the type is still fixed at compile time, so Java remains statically typed.
10. **What are checked and unchecked exceptions?** — Checked are compiler-enforced ( `IOException` ); unchecked ( `RuntimeException` ) aren't.
11. **finally — does it always run?** — Yes, except `System.exit()` or a JVM crash.
12. **try-with-resources?** — Automatically closes `AutoCloseable` resources when the block exits, even on exception.
13. **String immutability — why?** — Enables safe String Pool sharing, thread safety, security, and hashcode caching.
14. **String Pool?** — A special memory area caching string literals for reuse across equal literals.
15. **StringBuilder vs StringBuffer?** — StringBuilder is faster/unsynchronized; StringBuffer is synchronized (thread-safe, slower).
16. **What is a functional interface?** — An interface with exactly one abstract method, usable as a lambda target.
17. **Effectively final?** — A local variable assigned once and never reassigned — required for lambda capture.
18. **What is a Record?** — A concise, immutable data class (Java 16+) that auto-generates constructor/accessors/equals/hashCode/toString.
19. **What is a Sealed Class?** — Restricts which classes may extend/implement it via `permits`, enabling exhaustive pattern matching.
20. **Is Java 100% object-oriented?** — No — primitive types aren't objects, unlike purely OOP languages like Smalltalk.

## OOP (20)

21. **Four pillars of OOP?** — Encapsulation, Abstraction, Inheritance, Polymorphism.
22. **Encapsulation?** — Bundling data + methods, restricting access via modifiers to protect state.
23. **Abstraction?** — Hiding implementation complexity, exposing only essential features via abstract classes/interfaces.
24. **Abstraction vs Encapsulation?** — Abstraction hides complexity (design); encapsulation hides data (implementation).
25. **Inheritance?** — Child class reuses a parent's fields/methods; models an IS-A relationship.
26. **Why no multiple class inheritance in Java?** — Avoids the Diamond Problem's ambiguous method resolution.

27. **How do interfaces solve multiple inheritance?** — No conflicting state; Java 8+ forces explicit resolution of conflicting default methods.
28. **Polymorphism?** — One interface/name, many behaviors — via overloading (compile-time) and overriding (runtime).
29. **Dynamic Method Dispatch?** — JVM resolves an overridden method call based on the actual object type at runtime.
30. **Overloading vs Overriding?** — Overloading: same name, different params, compile-time. Overriding: same signature, subclass, runtime.
31. **Abstract class vs Interface?** — Abstract class allows partial implementation + state; interface is a pure contract (default/static methods since Java 8).
32. **Constructor chaining?** — Calling one constructor from another via `this()` (same class) or `super()` (parent).
33. **Can a constructor be private?** — Yes, commonly used in the Singleton pattern.
34. **What is the Object class?** — Root of all Java classes; provides `equals()`, `hashCode()`, `toString()`, `clone()`.
35. **equals()/hashCode() contract?** — Equal objects must have equal hashCodes, or hash-based collections break.
36. **Composition vs Inheritance?** — Composition (HAS-A, flexible) is often preferred over inheritance (IS-A, tightly coupled).
37. **Association vs Aggregation vs Composition?** — Increasing ownership strength: none → weak → strong (lifecycle-bound).
38. **What is SOLID?** — Five design principles for maintainable OOP: SRP, OCP, LSP, ISP, DIP.
39. **What is covariant return type?** — An overriding method can return a narrower (subtype) return type than the parent's.
40. **Static vs instance members?** — Static belongs to the class (one shared copy); instance belongs to each object.

## Collections (20)

41. **Why is Map not a Collection?** — It deals in key-value pairs, not single elements — a separate hierarchy entirely.
42. **ArrayList vs LinkedList?** — ArrayList: O(1) random access; LinkedList: O(1) end insert/delete, O(n) access.
43. **HashSet uniqueness — how?** — Backed by HashMap; uses hashCode()+equals() to detect duplicates.
44. **TreeSet backing structure?** — Red-Black Tree, keeps sorted order, O(log n) operations.
45. **Walk through HashMap.put() internals.** — Compute hash, find bucket, handle collision via chain/tree, insert or update.
46. **What is treeification?** — Bucket converts from linked list to red-black tree at 8+ collisions, bounding lookup to O(log n).
47. **HashMap load factor default?** — 0.75 — resize triggers when 75% full.
48. **HashMap vs Hashtable vs ConcurrentHashMap?** — HashMap unsynchronized; Hashtable fully locked, no nulls; ConcurrentHashMap fine-grained locked, no nulls.
49. **What happens if a mutable key is changed after insertion?** — The entry becomes unreachable — old bucket holds it, new hash points elsewhere.
50. **Fail-fast vs fail-safe iterators?** — Fail-fast throws ConcurrentModificationException on structural change; fail-safe tolerates it.
51. **Comparable vs Comparator?** — Comparable defines one natural ordering inside the class; Comparator defines external, unlimited orderings.
52. **PriorityQueue backing structure?** — A binary heap stored in an array.
53. **Why prefer ArrayDeque over Stack/LinkedList?** — Better performance, no unneeded synchronization, no per-node overhead.
54. **Collection vs Collections?** — Collection is an interface; Collections is a static utility class.

55. **What does Collections.unmodifiableList() do?** — Returns a read-only view, mutation attempts throw UnsupportedOperationException.
56. **What is type erasure?** — Generic type info is removed at compile time, replaced with Object/the bound type.
57. **What is PECS?** — "Producer Extends, Consumer Super" — the wildcard rule for generic method parameters.
58. **map vs flatMap in Streams?** — map is 1-to-1; flatMap transforms and flattens nested streams into one.
59. **groupingBy vs partitioningBy?** — groupingBy makes arbitrary groups; partitioningBy always makes exactly two (true/false).
60. **orElse vs orElseGet?** — orElse always evaluates eagerly; orElseGet evaluates lazily, only if empty.

## Java 8+ (15)

61. **Biggest Java 8 features?** — Lambdas, Streams API, Optional, default interface methods, new Date/Time API.
62. **What is lazy evaluation in Streams?** — Intermediate ops don't execute until a terminal operation triggers the pipeline.
63. **Intermediate vs terminal Stream operations?** — Intermediate (lazy, returns Stream) vs terminal (eager, triggers execution).
64. **What is short-circuiting?** — Operations like findFirst/anyMatch/limit can stop processing early.
65. **Can a Stream be reused?** — No — consumed once by a terminal operation; reuse throws IllegalStateException.
66. **Why was Optional introduced?** — To make "maybe absent" explicit in the type system, reducing accidental NPEs.
67. **Why avoid Optional as a field/parameter type?** — Not designed for that; intended purely as a method return type.
68. **What are method references?** — Shorthand lambda syntax that just calls an existing method, e.g. `String::length`.
69. **Why is java.time preferred over java.util.Date?** — Immutable, thread-safe, intuitive 1-indexed months, cleanly separated concerns.
70. **Duration vs Period?** — Duration = time-based amounts; Period = date-based amounts.
71. **What are Records for?** — Concise, immutable data-carrier classes with auto-generated boilerplate.
72. **What is pattern matching for instanceof?** — Auto-casts within the check: `if (obj instanceof String s)`.
73. **What is pattern matching for switch (Java 21)?** — Switch on an object's type/structure with destructuring support.
74. **What are Virtual Threads?** — JVM-managed lightweight threads enabling millions of concurrent I/O-bound tasks.
75. **What is the Collectors.groupingBy + counting pattern used for?** — Word/element frequency counting in one pipeline.

## Multithreading (15)

76. **Process vs Thread?** — Process has isolated memory; threads share memory within a process.
77. **Runnable vs Callable?** — Callable returns a value and can throw checked exceptions; Runnable can't.
78. **Thread lifecycle states?** — NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, TERMINATED.
79. **What does synchronized guarantee?** — Mutual exclusion AND happens-before visibility of prior writes.
80. **wait() vs sleep()?** — wait() releases the lock and needs synchronized context; sleep() doesn't release any lock.
81. **What does volatile guarantee?** — Visibility and ordering of a variable, NOT atomicity of compound operations.
82. **What is CAS?** — Compare-And-Set — an atomic hardware-supported operation underlying lock-free programming.

83. **Why do thread pools exist?** — Avoid the overhead of creating/destroying threads per task by reusing a bounded worker set.
84. **ReentrantLock vs synchronized?** — ReentrantLock offers tryLock, fairness, interruptible waits; synchronized is simpler, auto-releasing.
85. **How does ConcurrentHashMap avoid locking the whole map?** — Fine-grained per-bucket locking plus CAS operations.
86. **Four conditions for deadlock?** — Mutual exclusion, hold-and-wait, no preemption, circular wait.
87. **How do you prevent deadlock?** — Enforce a consistent global lock acquisition order.
88. **Why was CompletableFuture introduced?** — To support composable, non-blocking async pipelines, unlike plain Future's blocking-only get().
89. **What problem do virtual threads solve?** — Millions of lightweight, JVM-managed concurrent tasks without OS-thread-scale cost.
90. **What is the virtual thread "pinning" problem?** — Blocking inside `synchronized` prevents a virtual thread from unmounting its carrier.

## JVM & GC (10)

91. **Main JVM components?** — Class Loader Subsystem, Runtime Data Areas, Execution Engine.
92. **What replaced PermGen?** — Metaspace (Java 8+) — grows in native memory, avoiding PermGen's fixed-size OOM.
93. **Heap vs Stack?** — Heap is shared, stores objects; Stack is per-thread, stores locals/frames.
94. **What is a GC Root?** — A reachability starting point (stack locals, static fields, active threads, JNI refs).
95. **Minor GC vs Major GC vs Full GC?** — Minor = Young Gen; Major = Old Gen; Full = entire heap, slowest.
96. **Default GC since Java 9?** — G1 (Garbage First) — region-based, balances throughput and pause time.
97. **What causes StackOverflowError?** — Excessive/unterminated recursion exhausting a thread's stack.
98. **How can a memory leak happen with automatic GC?** — An object stays reachable (e.g., unbounded static collection) even when logically unneeded.
99. **What is "happens-before"?** — The JMM's guarantee that if A happens-before B, A's memory effects are visible to B.
100. **Why is BigDecimal preferred over double for money?** — double can't precisely represent many decimal fractions, causing unacceptable rounding errors in financial math.

# Part 16 — HR + Technical Bridge Questions

*(These sit between HR and technical rounds — answer with real specifics from your own project, not generic textbook phrasing.)*

**"Explain your Java project."**

- **Structure your answer:** (1) One-line purpose, (2) your architecture (layers, tech stack), (3) one interesting technical challenge you solved, (4) the outcome/impact.
- **Example shape:** "I built an order management REST API using Spring Boot. It follows a Controller-Service-Repository architecture with PostgreSQL. The interesting part was handling concurrent stock updates — I used optimistic locking with a

`@Version` field to prevent overselling under load. It now handles X requests/sec in production."

**"Why Java?"**

- Mature ecosystem (Spring, Hibernate), strong typing catches bugs early, huge talent pool and community support, excellent performance via JIT, and it remains dominant in enterprise/backend systems — a safe, well-supported long-term choice for backend work.

**"Difference between academic and real project code?"**

- Academic code optimizes for correctness on a known input; real code must handle unknown/malicious input (validation), concurrent access, partial failures (network/DB errors), logging/observability, and long-term maintainability by a team — not just "does it produce the right answer once."

**"What Java feature do you use most?"**

- Give a specific, honest answer tied to your project — e.g., "Streams for transforming entities to DTOs," or "Optional to avoid null checks in my service layer," or "CompletableFuture for parallelizing independent API calls." Avoid vague answers like "OOP" — be concrete.

**"Explain a bug you fixed."**

- **Structure:** (1) Symptom observed, (2) how you diagnosed it (logs, debugger, thread dump), (3) root cause, (4) the fix, (5) how you verified/prevented recurrence.
- **Good example topics from this series:** a `ConcurrentModificationException` from removing during iteration, a `NullPointerException` from an unhandled `Optional`, or a race condition fixed with proper synchronization.

**"Explain a performance improvement you made."**

- **Good example shapes:** "Replaced String concatenation in a loop with StringBuilder," "Added an index + used PreparedStatement batch updates instead of individual inserts," "Pre-sized a HashMap to avoid repeated rehashing," or "Introduced caching for a frequently-read, rarely-changed dataset."

**"Explain thread safety in your project."**

- Identify a **specific shared mutable resource** (a cache, a counter, a shared connection) and explain the exact tool you used to protect it — `synchronized`, `ConcurrentHashMap`, `AtomicInteger`, or a DB-level lock — and *why* that specific tool fit the situation better than the alternatives.

# Part 17 — 7-Day Java Revision Plan

| Day | Focus | What to Revise |
|---|---|---|
| **Day 1** | Foundations & OOP (Handbook 1) | JVM basics, all 4 pillars, constructors, keywords — do the Top 25 questions |
| **Day 2** | Core Java Essentials (Handbook 2) | Exceptions, Strings, Generics (PECS!), IO/NIO, Date-Time, Reflection/Annotations |
| **Day 3** | Collections Deep-Dive (Handbook 3, Part 1) | HashMap internals cold, ArrayList/LinkedList, Set/Map family, Comparable/Comparator |
| **Day 4** | Modern Java (Handbook 3, Part 2) | Streams, Collectors, Optional, Records/Sealed Classes/Pattern Matching |
| **Day 5** | Concurrency & JVM (Handbook 4) | synchronized/volatile, Executor/CompletableFuture, Virtual Threads, JVM/GC basics |
| **Day 6** | Backend & Spring Boot (Handbook 5, Parts 1–11) | DTO vs Entity, exception handling, JDBC/PreparedStatement, real architecture |
| **Day 7** | Full Mock Interview Day | Do Part 12 scenarios out loud, Part 13 output prediction, Part 14 debugging round, then the Top 100 (Part 15) rapid-fire |

**How to use each day:** Spend the first ~70% of study time re-reading and doing the practice problems, and the last ~30% purely on that handbook's Interview Questions sections and cheat sheets — recall practice matters more than re-reading.

# Part 18 — Last-Minute Placement Revision

## One-Page Java Master Cheat Sheet

- **JDK ⊃ JRE ⊃ JVM.** Bytecode is portable; the JVM itself is platform-specific.
- **4 Pillars:** Encapsulation (hide data), Abstraction (hide complexity), Inheritance (IS-A reuse), Polymorphism (many forms).
- **Overloading** = compile-time, same class. **Overriding** = runtime, subclass, dynamic dispatch.
- **String** is immutable — String Pool caches literals; `new String()` bypasses it; always use `.equals()`.
- **HashMap:** hash → bucket → chain/tree; load factor 0.75; treeifies at 8+ collisions.
- **Streams** are lazy until a terminal op; `map` = 1-to-1, `flatMap` = flatten; `groupingBy` vs `partitioningBy`.
- **Optional:** return type only; `orElseGet` for expensive defaults, never `orElse`.
- `synchronized` gives mutual exclusion + visibility; `volatile` gives visibility only, not atomicity.
- **Deadlock's 4 conditions:** mutual exclusion, hold-and-wait, no preemption, circular wait.

- **Virtual threads:** JVM-managed, cheap, unmount on blocking I/O; `synchronized` pins the carrier.
- **G1** is the default GC since Java 9 — region-based, balances throughput and pause time.
- **BigDecimal**, never `double`, for money. **PreparedStatement**, never string concat, for SQL.
- **DTO ≠ Entity** — DTOs protect security, performance, and API stability.
- **Constructor injection** > field injection in Spring — enables `final`, easier testing.

## Collections Cheat Sheet

| Structure | Order | Key Trait |
|---|---|---|
| ArrayList | Insertion | O(1) random access |
| LinkedList | Insertion | O(1) end insert/delete |
| HashSet/HashMap | None | O(1) avg, hash-based |
| LinkedHashSet/Map | Insertion | HashMap + linked list |
| TreeSet/TreeMap | Sorted | O(log n), Red-Black Tree |
| ArrayDeque | Insertion | Modern stack/queue choice |
| PriorityQueue | Heap order | Binary heap, O(log n) |

## Streams Cheat Sheet

- Intermediate (lazy): `map`, `filter`, `flatMap`, `distinct`, `sorted`, `limit`, `skip`, `peek`
- Terminal (eager): `collect`, `forEach`, `reduce`, `count`, `findFirst`, `anyMatch` / `allMatch` / `noneMatch`
- `Collectors.groupingBy(classifier, downstream)` for multi-group aggregation
- `Collectors.partitioningBy(predicate)` for true/false split
- Short-circuit ops (`findFirst`, `anyMatch`, `limit`) can stop early — safe even on infinite streams

## Concurrency Cheat Sheet

- `Runnable` (no return) vs `Callable` (returns value, throws checked exceptions)
- `ExecutorService.submit()` → `Future`; always `shutdown()` when done
- `ReentrantLock` > `synchronized` when you need `tryLock`, fairness, or interruptible waits
- `ConcurrentHashMap`: fine-grained locking, no nulls; `CopyOnWriteArrayList`: read-heavy only
- `CompletableFuture`: `thenApply` (transform), `thenCompose` (chain async), `thenCombine` (merge two)
- Virtual threads: millions possible, great for I/O-bound concurrency, not CPU-bound speed

## JVM Cheat Sheet

- Heap (shared, objects) / Stack (per-thread, frames) / Metaspace (class metadata, native memory)
- Class loading: Loading → Verification → Preparation → Resolution → Initialization
- Minor GC (Young Gen, frequent) vs Major/Full GC (Old Gen/whole heap, rare, slow)
- GC Root reachability determines what's alive, not reference counting
- `happens-before`: program order, unlock→lock, volatile write→read, thread start/join

## Spring Boot Connection Sheet

| Spring Concept | Core Java Behind It |
|---|---|
| Dependency Injection | Composition (HAS-A, injected not `new`-ed) |
| IoC Container | Reflection creates/wires objects at startup |
| `@Component` / `@Service` / `@Repository` | Annotations read via Reflection |
| `Repository<T,ID>` | Generics |
| `findById()` → `Optional<T>` | Optional as a return type |
| DTO mapping | Streams `.map()`, immutability (Records) |
| `@RestControllerAdvice` | Centralized custom exception handling |
| `@Transactional` | Ties into JDBC connection/transaction management |

## Top 25 Must-Revise Concepts

1. HashMap internal working (put/get, hashing, collisions, treeification)
2. Four pillars of OOP with real examples
3. Overloading vs Overriding (compile-time vs runtime)
4. String immutability and the String Pool
5. `==` vs `.equals()` for Strings and wrapper classes
6. Checked vs unchecked exceptions
7. try-with-resources and suppressed exceptions
8. Generics: PECS and type erasure
9. Streams: lazy evaluation, map vs flatMap
10. Collectors: groupingBy vs partitioningBy
11. Optional: correct usage, orElse vs orElseGet

12. Records and Sealed Classes (Java 17)
13. Pattern matching for switch (Java 21)
14. synchronized vs volatile vs Atomic classes
15. Deadlock's four conditions and prevention
16. ExecutorService and thread pool types
17. CompletableFuture chaining (thenApply/thenCompose/thenCombine)
18. Virtual Threads and the pinning problem
19. JVM architecture (Heap/Stack/Metaspace/Execution Engine)
20. Garbage Collection (Minor/Major/Full, G1)
21. Java Memory Model and happens-before
22. DTO vs Entity and why they're separated
23. PreparedStatement and SQL injection prevention
24. Dependency Injection and constructor injection in Spring
25. BigDecimal for financial calculations

## Top 25 Must-Revise Interview Questions

1. Explain JDK vs JRE vs JVM.
2. Walk through HashMap.put() internals step by step.
3. Why is String immutable?
4. Difference between abstraction and encapsulation?
5. Why doesn't Java support multiple inheritance with classes?
6. Explain dynamic method dispatch with an example.
7. Why override hashCode() when you override equals()?
8. What is PECS and when do you use it?
9. What is type erasure and its consequences?
10. Explain lazy evaluation in Streams and why it matters.
11. groupingBy vs partitioningBy — when would you use each?
12. orElse vs orElseGet — what's the performance difference?
13. What does volatile guarantee, and what does it NOT guarantee?
14. What are the four necessary conditions for deadlock?
15. thenApply vs thenCompose in CompletableFuture?
16. What problem do virtual threads solve, and what's the pinning issue?
17. Explain Minor GC vs Major GC vs Full GC.
18. What is "happens-before" in the Java Memory Model?

19. Why use a DTO instead of returning the Entity directly?
20. How does PreparedStatement prevent SQL injection?
21. Why is constructor injection preferred in Spring?
22. How would you handle concurrent stock updates in an order system?
23. Why should an API never expose a raw stack trace?
24. Why use BigDecimal instead of double for money?
25. Explain a bug you personally fixed, step by step.

## 15-Minute Emergency Revision

If you have only 15 minutes before walking into an interview, read these in order:

1. **(3 min)** One-Page Java Master Cheat Sheet (above)
2. **(3 min)** HashMap internals — put()/get() flow, load factor, treeification
3. **(2 min)** Streams cheat sheet — map vs flatMap, groupingBy vs partitioningBy
4. **(2 min)** Concurrency cheat sheet — synchronized vs volatile, deadlock's 4 conditions
5. **(2 min)** Spring Boot Connection Sheet — DI, DTO vs Entity, constructor injection
6. **(3 min)** Skim the Top 25 Must-Revise Interview Questions and mentally rehearse one-sentence answers

**Final reminder:** Interviewers reward **explaining WHY**, not reciting definitions. For every answer, be ready to add one sentence of *reasoning* — "...and the reason that matters is ___." That single habit is what separates a memorized answer from a confident, senior-sounding one.

## Series Complete 🎓

This concludes the **Java Placement Master Series**:

| Handbook | Focus |
|---|---|
| 1 | Java Foundations & Object-Oriented Programming |
| 2 | Core Java Essentials (Exceptions, Strings, Generics, IO/NIO, Reflection) |
| 3 | Collections Framework & Modern Java (Streams, Optional, Java 8–21) |
| 4 | Multithreading, JVM, Garbage Collection & Performance |
| 5 | Backend Java & Interview Mastery (this handbook) |

You now have a complete, placement-ready Java knowledge base — from JVM internals to production Spring Boot patterns. Revisit the cheat sheets in each handbook's final section the night before every interview, and good luck!